

packages/graph-core/scripts/generate-alphabetical-catalog.mjs

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T19:23:06.565Z

Overview

A new Node script `packages/graph-core/scripts/generate-alphabetical-catalog.mjs` (added lines R1-323) reads `language-names.txt`, normalizes language names, resolves tree-sitter WASM binaries and file extensions, and writes a sorted `alphabetical-catalog.generated.ts` module.

Key changes

- **Imports** (R6-8): `readFileSync`, `writeFileSync` from `node:fs`; `dirname`, `join` from `node:path`; `fileURLToPath` from `node:url`; and a type import `LanguageCatalogEntry` from `./catalog-types.js` (R313).
- **Constants**: `WASMS_BY_KEY` and `EXT_BY_KEY` map language identifiers to WASM files and extensions.
- **Normalization helpers**:
 - `normalizeKey(name)` (lines 182-214) trims and normalizes display names, mapping common aliases.
 - `slugify(name, used)` (lines 217-230) creates a URL-friendly id, ensuring uniqueness via a `Set`.
 - `resolveWasmsKey(normKey, id)` (lines 233-260) selects the correct WASM key, using an alias table.
 - `resolveExtensions(normKey, displayName)` (lines 263-270) returns extensions, falling back to substring matching.
- `claimExtensions(exts, wasmsFile)` (lines 282-290) guarantees each extension appears only once across all entries.
- **Main loop** (lines 293-300) builds `{ id, displayName, extensions, wasmsFile }` objects, sorts them alphabetically, and writes the output file with a header comment and an exported `alphabeticalCatalogEntries` array (export added at R316).

Impact

- The script must run before any build that imports `alphabetical-catalog.generated.ts`.
- Unique ids and deduplicated extensions reduce potential runtime errors in language-specific tooling.
- Processing is linear in the number of language names; the added step is negligible compared to the overall build.
- Console logs (lines 321-323) report the total number of languages and how many have WASM support, aiding debugging.

Risks & follow-ups

- A missing or malformed `language-names.txt` will cause the script to crash; verify the file's presence and format.
- Errors in the alias table could map a language to an incorrect WASM or extension; run unit tests covering a representative set of languages.
- `slugify` uses a `Set`, but a future language with the same slug could silently overwrite; monitor the console output for duplicate id warnings.
- Ensure the generated file is included in the TypeScript build and that consumers import the correct path.